Bayesian Handwritten Digit Recognition

Last-Layer Laplace Inference with an Exact Conditional Hessian

Paco, Sou Chak Tong
Student ID: U-C3-2652-3
CISC3024 — Pattern Recognition
AI Assignment #2

8 October 2026

Abstract

This coursework implements a small-scale last-layer Laplace classifier motivated by *Laplace Redux* (NeurIPS 2021). A learned image representation is frozen, a Gaussian prior is placed over its linear classification head, and an exact conditional softmax Hessian defines a Gaussian approximation to the posterior. Predictions average class probabilities over 512 sampled heads. The study uses 1,797 handwritten digit images, a fixed stratified split of 1,077/360/360 images, and three independent neural initialization seeds. Validation selects checkpoints and prior precision before final test prediction. Laplace achieves 97.69% mean accuracy and 0.151 mean negative log-likelihood (NLL). A separately selected maximum a posteriori (MAP) classifier has lower NLL (0.099). A frozen-prior ablation confirms that posterior averaging reduces probability quality here, while dropping posterior correlations reduces it further. Thus Bayesian inference provides explicit uncertainty, but does not guarantee better calibration. The AI agent performed research, programming, experiments and report drafting; the personal reflection remains student-authored.

Scope. This is an independently coded, educational adaptation of a recent practical Bayesian framework, not a reproduction of its published benchmark results. All numerical results come from this project's saved outputs. Source code and evidence are publicly available at the verified repository linked in Section 6.

Contents

1 How AI Found the Algorithm **2**

2 Algorithm Description **5**

3 How AI Implemented the Algorithm **8**

4 Experiment Settings and Results **9**

5 What I Have Learnt from This AI Assignment **18**

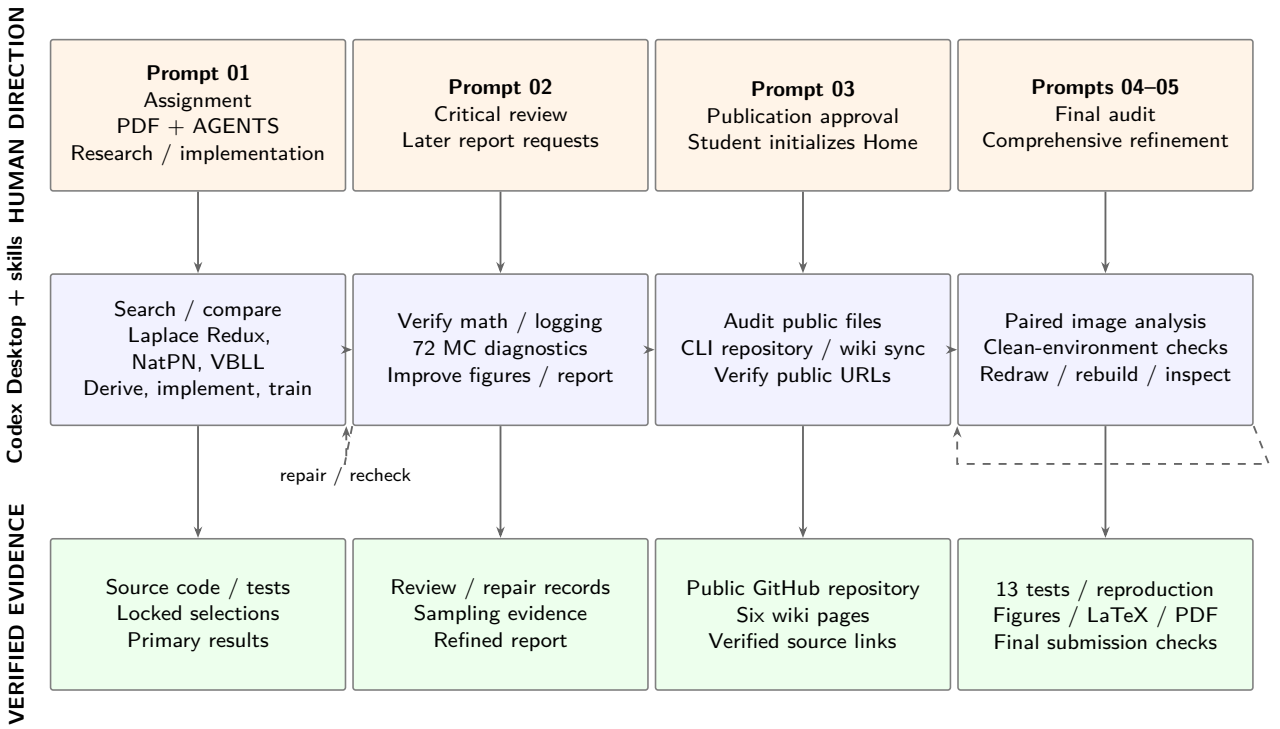
6 Webpage Link of the Source Codes **19**

1 How AI Found the Algorithm

1.1 How the student structured Codex's work

The student guided Codex Desktop through a hierarchy of written instructions. The instructor's `AIassign2.pdf` defined the assignment: a recent probability- or Bayes-based recognition algorithm, AI-written technical code and six required report items. Codex extracted the complete one-page PDF and inspected its rendering. The project `AGENTS.md` supplied persistent rules for scientific integrity, reproducibility, mathematical rigor, held-out evaluation and student authorship. Task-specific Markdown prompts supplied the executable stages; installed skills supplied procedural guidance for carrying out those stages. The project priority was course requirements, then `AGENTS.md`, then the current task prompt, then skill guidance.

The student explicitly invoked Prompt 01 for research, implementation, experiments and reporting, then Prompt 02 for technical and figure review. Later requests refined the report and supplied the student's own reflection. Workflow diagram 1 distinguishes these executed activities and the public repository, including the final submission audit. Descriptions below are evidence-based paraphrases; only the quotation column reproduces prompt text.



Workflow diagram 1. Executed instruction-to-evidence stages. Human direction (orange), Codex activities (blue) and outputs (green) remain distinct. Research/design skills supported comparison and controls; ML/PyTorch supported implementation; statistics, visualization and LaTeX/PDF guidance supported review and refinement. Dashed arrows denote same-agent repair and rechecking, not independent human review. Prompt 05 retains the selected models and reflection while extending analysis and verifying the deliverables.

What the skills actually contributed. The usage record identifies `literature-review` for primary-source provenance; `experimental-design` for matched controls and seed replication; and `scikit-learn` for splitting, baselines and metric conventions. Machine-learning, `pytorch-patterns` and `pytorch-training-recipe` guidance informed deterministic training, device handling and checkpoint checks. `scientific-visualization` informed scales, sample counts, vector exports and final-size readability. PDF guidance supported extraction/rendering, while the LaTeX compile plugin provided compiler detection and actual builds. These were procedural aids, not experimental evidence [8].

`scientific-writing` and `latex-paper-en` were inspected in Prompt 01, but their review services/scripts were not used then. Prompt 02 materially applied their writing, caption and reviewer guidance, together with `statistical-analysis` for replication units and uncertainty interpretation. Installed `pymc`, `exploratory-data-analysis`, `analytical-method-validation` and `markdown-mermaid-writing` have no documented material role and are not credited as used. No subagent, paid schematic generator or external manuscript-review service was used.

1.2 Verified instructions and their executed responses

P01 denotes `prompts/01_RESEARCH_IMPLEMENT_REPORT.md`; P02 denotes `prompts/02_CRITICAL_REVIEW_AND_FIGURES.md`. A, C and D identify the named phases in P01; QA3 is step 3 of P02's required QA loop. Each row gives a verbatim extract, its purpose and the recorded response. The comparison extract begins mid-sentence; none of the extracts is a fabricated conversation or a claim that every available skill was invoked.

Source	Verbatim instruction	Purpose and actual response
P01/D	"Use only verified citations."	**Source verification.** Primary proceedings/manuscripts supplied metadata; queries and source URLs were recorded, and all eight references were rechecked in Prompt 02.
P01/A	"and compare: publication/year, mathematical basis, relevance to the course, reproducibility, compute/data requirements, evaluation protocol, and risk."	**Substantive comparison.** Codex compared Laplace Redux, NatPN and VBLL on these criteria before implementing conditional Laplace.
P01/A	"Prefer genuinely recent publications (approximately 2021–2026) while citing older foundational Bayesian/probabilistic work when appropriate."	**Honest recency.** The report identifies the 2021 choice as the older edge of this range, explains its practical contribution and treats MacKay as foundational background.
P01/A	"Verify important symbolic or numerical identities against the implementation."	**Mathematical accountability.** Analytic gradients/Hessians were checked against autograd; covariance sampling, normalization and a two-class example were tested.
P01/C	"Use a proper train/validation/test strategy; select models with validation only."	**Leakage prevention.** Saved split indices and selection records document validation-based checkpoint/prior choice before final test prediction, with three neural seeds.
P02/QA3	"Re-run affected tests, figure generation, experiments where necessary, and bibliography/reference checks."	**Repair with evidence.** The loss-logging repair was tested and reproduced; nine tests passed, 18 primary arrays stayed identical, and citations were rechecked.
P02/Figures	"Use a controlled layout with informative whitespace; no unnecessary pages or enormous gaps."	**Final-size readability.** Codex enlarged labels, redesigned reliability/uncertainty displays and inspected exports on rendered report pages; later refinement corrected bounded F1 presentation.

Table 1: Selected instructions verified against the original prompt files. Responses are summaries of saved research, implementation and review records.

Recorded discovery queries (8 October 2026). The research log retains these actual algorithm-search strings:

- *Laplace Redux Effortless Bayesian Deep Learning NeurIPS 2021 Daxberger*
- *Posterior Network uncertainty estimation without OOD samples NeurIPS 2020 Natural Posterior Network 2021*
- *2022 Bayesian neural network classification variational inference last layer uncertainty paper*

The second query includes an older Posterior Network search term; the candidate actually compared was Natural Posterior Network (ICLR 2022). Additional recorded queries targeted Guo's calibration paper and MacKay's Bayesian interpolation paper for background, rather than treating them as recent candidate methods.

Verification and scope. Codex opened primary proceedings, author manuscripts and dataset documentation, checking titles, authors, venues/years and available DOI/page metadata. Laplace's NeurIPS proceedings/PDF, NatPN's ICLR PDF/arXiv manuscript and VBLL's ICLR PDF/arXiv record supplied the candidate evidence. OpenReview forum pages returned a verification challenge and a NeurIPS BibTeX link returned a tool error; accessible primary PDFs/records supplied the missing metadata. This was a bounded narrative search, not an exhaustive systematic review. Verification was by the AI agent, not an independent human reviewer.

1.3 Candidate comparison and final selection

Method	Publication	Defining probability mechanism	Feasibility and main risk
Laplace Redux	NeurIPS 2021	Gaussian approximation around MAP; posterior integration	Post-hoc head inference is tractable; a local Gaussian can misrepresent uncertainty.
Natural Posterior Network	ICLR 2022	Density-dependent conjugate updates for exponential-family targets	Learned features and normalizing flows; density training adds complexity and failure modes.
Variational Bayesian Last Layers	ICLR 2024	Deterministic variational formulation for Bayesian output layers	Efficient last-layer training; its variational objective adds implementation work.

Table 2: Candidate methods verified from primary sources [1, 2, 3]. All address probabilistic prediction and can be evaluated on image recognition.

Why Laplace Redux rather than a newer method? The original choice prioritized an auditable probability model and controlled approximation comparisons. Section 2 of Laplace Redux separates the uncertain weight subset, curvature structure, prior selection and predictive integration [1]. These choices directly support this study's fixed-feature, matched-MAP and diagonal-precision controls. With only 330 head parameters, the complete conditional Hessian can be derived, stored and checked against autograd; linear head logits also make the conditional Hessian equal to the generalized Gauss–Newton curvature. This gives a concrete route from prior and likelihood to a testable posterior calculation.

Newer alternatives remain credible. VBLL offers efficient sampling-free variational training and explicitly supports frozen-feature post-training [3]; it was not rejected as infeasible. Its learned variational mean/covariance and bound-based objective would change the inference question from local curvature at a conditional MAP optimum. NatPN instead converts learned feature densities into conjugate posterior updates [2], adding a density estimation mechanism suited to a different uncertainty study. The preference here is mathematical inspectability, not an empirical claim that these methods are less accurate or less calibrated; neither was run in this project.

The 2021 paper is at the older edge of the requested 2021–2026 range. Its recent contribution is a practical framework/library and evaluation, not the invention of Bayes or Laplace theory [4]. This implementation chooses full curvature, validation tuning and Monte Carlo prediction instead of the paper's default KFAC, empirical-Bayes and probit choices [1]. The justification therefore supports an educational adaptation, not a state-of-the-art or full benchmark-reproduction claim, and does not use test outcomes to change the method.

Recency boundary. In October 2026 this is a five-year-old framework. The instructor supplies no numerical cutoff; Prompt 01 specifies approximately 2021–2026. The examiner may interpret recency more strictly. No test-driven replacement, superiority over unexecuted candidates or state-of-the-art claim is made.

1.4 How staged review changed the outcome

Prompt 01 produced the executed study and report. Prompt 02 strengthened the exact-Hessian/approximate-posterior distinction, tested a binary covariance example and repaired completed-epoch loss logging; all primary arrays remained identical. Its 72 fixed-model predictions quantified MC integration variation without retuning or treating seed spread as a dataset interval. Review also enlarged labels, separated seed markers and exposed calibration-bin support and unequal correct/error groups. Subsequent requests preserved all F1 observations and improved layout. Prompt 05 replaces the deficit display with paired signed F1 differences, adds image-bootstrap and sparse-bin intervals, redesigns both workflows/all exports, and verifies a clean environment. Nature Figure/composition, statistical, LaTeX and PDF guidance supported this same-agent refinement; no external peer review or gallery/generative service is claimed.

Complete records. The public repository in Section 6 contains the assignment, `AGENTS.md`, original `prompts/`, research/AI-use/review records, verified excerpts and `docs/skill_sources.md` with consultation, license, source links and hashes. Prompt 03 published after explicit student approval; the student's initial Home page enabled CLI synchronization of six wiki pages. Prompt 04 audited submission; Prompt 05's current evidence is in `docs/refinement_05.md`. Full technical details follow in Sections 2–4.

2 Algorithm Description

2.1 Task, notation and conditional model

An image is an 8×8 grayscale array with integer acquisition values from 0 to 16. Flattening gives $x \in [0,1]^{64}$ after division by the fixed maximum 16. Let $y \in \{0, \ldots, 9\}$ be its digit label, $K = 10$, and $\mathcal{D} = \{(x_n, y_n)\}_{n=1}^{N}$ the training partition. A deterministic neural representation is

$$h_\phi(x) = \tanh(Ax + a) \in \mathbb{R}^{32}, \qquad f_\phi(x) = \begin{bmatrix} h_\phi(x) \\ 1 \end{bmatrix} \in \mathbb{R}^{D}, \quad D = 33. \tag{1}$$

The appended one makes bias parameters part of the same algebra as weights. A head $W \in \mathbb{R}^{K \times D}$ produces logits $z_k = w_k^\top f_\phi(x)$. After training the representation, ϕ is frozen. Bayesian inference is *conditional on this learned representation*; it does not infer a distribution over all network weights.

Symbol	Meaning
$N = 1077$	Training images
$K = 10$, $D = 33$	Classes and augmented feature dimension
$\theta = \mathrm{vec}_{\mathrm{row}}(W) \in \mathbb{R}^{330}$	Class-major head parameter vector
$\alpha > 0$	Isotropic Gaussian prior precision
H, Σ	Posterior precision Hessian and covariance
$S = 512$	Monte Carlo weight draws per prediction run
$q(\theta)$	Gaussian approximation to the conditional posterior

Table 3: Notation matched to array layouts in `laplace.py`.

2.2 Likelihood, prior and posterior

The defining conditional likelihood is categorical:

$$p(y = k \mid x, W, \phi) = p_k(x; W) = \frac{\exp(w_k^\top f_\phi(x))}{\sum_{j=0}^{K-1} \exp(w_j^\top f_\phi(x))}. \tag{2}$$

Training observations are modeled as conditionally independent. An isotropic prior is imposed on *every* head parameter, including biases:

$$p(\theta \mid \alpha) = \left(\frac{\alpha}{2\pi}\right)^{KD/2} \exp\left(-\frac{\alpha}{2}\|\theta\|_2^2\right). \tag{3}$$

Bayes' rule gives

$$p(\theta \mid \mathcal{D}, \phi, \alpha) = \frac{p(\mathcal{D} \mid \theta, \phi) p(\theta \mid \alpha)}{\int p(\mathcal{D} \mid u, \phi) p(u \mid \alpha)\, du}. \tag{4}$$

For fixed α and ϕ, constants independent of θ can be discarded. The negative log-posterior objective is therefore

$$L(\theta) = \sum_{n=1}^{N} \left[\log \sum_j \exp(w_j^\top f_n) - w_{y_n}^\top f_n\right] + \frac{\alpha}{2}\|\theta\|_2^2. \tag{5}$$

The implementation uses stable `logsumexp` and sums the likelihood. A mean loss with the same numerical α would change the prior's relative strength by a factor of N. Feature training uses a separate mean cross-entropy and regularizer; its optimizer decay is not identified with this conditional head prior.

2.3 MAP estimation and exact curvature

For fixed features, the objective is convex. A strictly positive prior precision makes it strictly convex, with a unique head optimum $\hat{\theta} = \arg\min_\theta L(\theta)$. Let $t_{nk} = \mathbb{1}\{y_n = k\}$. Differentiating the normalizer gives $\partial \log \sum_j e^{z_j} / \partial z_k = p_{nk}$, hence

$$\frac{\partial L}{\partial w_k} = \sum_{n=1}^{N} (p_{nk} - t_{nk}) f_n + \alpha w_k. \tag{6}$$

L-BFGS minimizes this objective using an analytic gradient. Every run checks the optimizer status and requires the maximum absolute summed-objective gradient component to be below 10^{-3}. Recorded candidate gradients are much smaller.

Differentiating softmax gives $\partial p_k/\partial z_l = p_k(\delta_{kl} - p_l)$. Thus the exact Hessian entries, in the class-major ordering used by the code, are

$$H_{(k,a),(l,b)} = \sum_{n=1}^{N} p_{nk}(\delta_{kl} - p_{nl}) f_{na} f_{nb} + \alpha \delta_{kl} \delta_{ab}. \tag{7}$$

Equivalently, with $B_n = \operatorname{diag}(p_n) - p_n p_n^\top$,

$$H = \sum_{n=1}^{N} B_n \otimes (f_n f_n^\top) + \alpha I_{KD}. \tag{8}$$

This is the *exact conditional* Hessian for linear logits. It is not an exact full-network Hessian: nonlinear feature parameters are held fixed. A generalized Gauss–Newton expression coincides with this Hessian in the conditional linear-logit case.

For any $v \in \mathbb{R}^K$, $v^\top B_n v = \sum_k p_{nk} v_k^2 - (\sum_k p_{nk} v_k)^2 \geq 0$: it is a variance under a categorical distribution. Thus $H \succeq \alpha I$ and Cholesky factorization is well defined. At $\alpha = 0$, a common-logit shift direction is unidentifiable and curvature is singular. A proper prior resolves this issue. Nonpositive precisions are rejected rather than hidden by arbitrary damping.

2.4 Local Gaussian approximation

Expand the negative log-posterior near its optimum:

$$L(\theta) \approx L(\hat{\theta}) + \underbrace{\nabla L(\hat{\theta})^\top (\theta - \hat{\theta})}_{\approx 0} + \frac{1}{2}(\theta - \hat{\theta})^\top H (\theta - \hat{\theta}). \tag{9}$$

Exponentiating its negative and normalizing yields

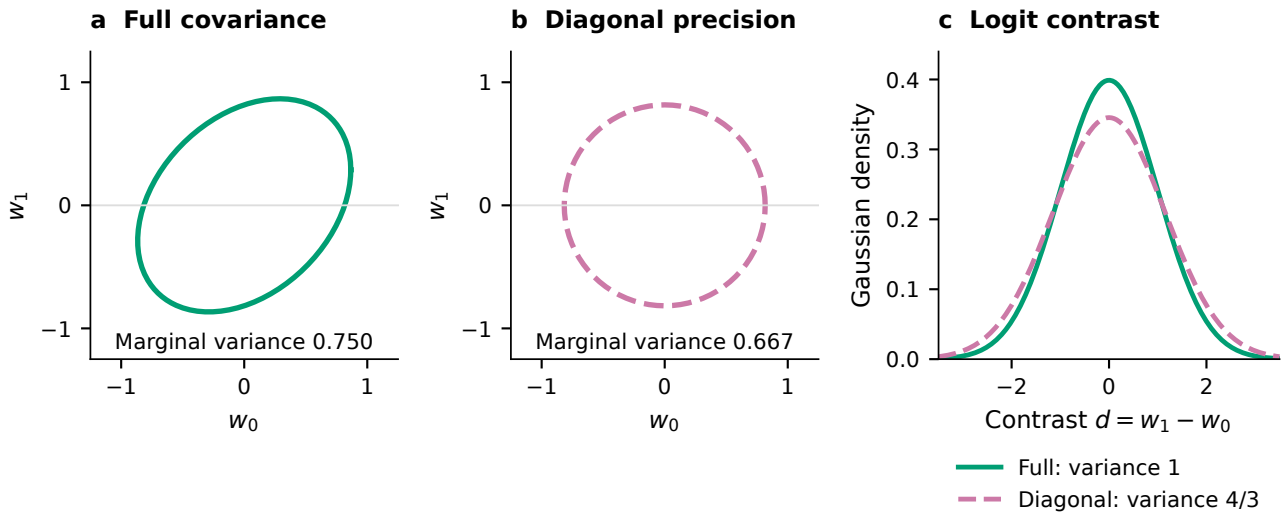
$$q(\theta) = \mathcal{N}(\hat{\theta}, H^{-1}), \qquad \Sigma = H^{-1}. \tag{10}$$

A Gaussian prior is not conjugate to a categorical softmax likelihood; the true posterior is generally not Gaussian. The second-order expansion is an approximation even when its Hessian is exact. The code computes the full 330×330 precision matrix in double precision. No diagonal or Kronecker curvature approximation is used for the primary method. The small problem makes its full covariance practical and indexing testable.

Worked conditional example. Consider two classes, one scalar feature $f = 1$, two training observations with labels 0 and 1, and $\alpha = 1$. Symmetry and strict convexity give $\hat{W} = (0, 0)^\top$ and $p = (1/2, 1/2)$. Each observation contributes $B = \frac{1}{4}\left[\begin{smallmatrix} 1 & -1 \\ -1 & 1 \end{smallmatrix}\right]$; summing both contributions and adding the prior gives

$$H = \begin{bmatrix} 1.5 & -0.5 \\ -0.5 & 1.5 \end{bmatrix}, \qquad \Sigma = H^{-1} = \begin{bmatrix} 0.75 & 0.25 \\ 0.25 & 0.75 \end{bmatrix}. \tag{11}$$

The probability depends on the logit difference $d = w_1 - w_0$, not their common offset. Its variance is $0.75 + 0.75 - 2(0.25) = 1$. Dropping off-diagonal precision first gives covariance $(2/3)I$ and $\operatorname{Var}(d) = 4/3$. Thus the diagonal approximation can increase *contrast* variance even though each marginal variance decreases. The common direction $(1, 1)$ has curvature 1, supplied solely by the prior; the contrast direction $(1, -1)$ has curvature 2. A unit test verifies the MAP, Hessian and both contrast variances. This exact toy calculation illustrates correlation cancellation; it does not measure the ten-class experiment's mechanism.



Worked-example geometry (not experimental data). Radius-one Mahalanobis contours of the two Gaussian approximations have the same center. Diagonalizing precision before inversion shrinks marginal variances from 3/4 to 2/3, yet the contrast $w_1 - w_0$ becomes wider: variance 1 versus 4/3. These contours are not 68% joint credible regions; the exact Gaussian contrast densities at right explain why smaller marginal variance need not mean more certain class probabilities. The contour equation is verified numerically.

2.5 Posterior predictive classification

A Bayesian prediction integrates the uncertain head:

$$\bar{p}_k(x) = \int p_k(x;\theta) q(\theta)\, d\theta \approx \frac{1}{S}\sum_{s=1}^{S} p_k(x;\theta^{(s)}), \qquad \hat{y}(x) = \arg\max_k \bar{p}_k(x). \tag{12}$$

Let $H = LL^\top$ with lower-triangular L. For $\epsilon^{(s)} \sim \mathcal{N}(0, I)$, sampling uses a triangular solve:

$$\theta^{(s)} = \hat{\theta} + L^{-\top}\epsilon^{(s)}, \qquad \mathrm{Cov}(L^{-\top}\epsilon) = L^{-\top}L^{-1} = H^{-1}. \tag{13}$$

The transpose matters: $L^{-1}\epsilon$ generally gives the wrong covariance. A simulation test checks sample covariance against a known non-diagonal precision inverse. The same 512 sampled heads predict all images within one call, representing global parameter uncertainty.

Validation draws use seed $r+1000$ and test draws $r+2000$ for initialization seed $r \in \{11, 22, 33\}$. Averaged class probabilities remain normalized. In general $\mathbb{E}[\mathrm{softmax}(z)] \neq \mathrm{softmax}(\mathbb{E}[z])$, so MAP and Laplace may disagree on probabilities even when their class predictions agree. For intuition, two illustrative equally weighted logit differences 0 and 4 give mean class-1 probability $(0.5 + 0.9820)/2 = 0.7410$, whereas the probability at their mean difference 2 is 0.8808. These chosen values illustrate nonlinear averaging, not actual Gaussian draws or dataset results.

2.6 Uncertainty and interpretation

Predictive entropy summarizes total uncertainty:

$$\mathcal{H}(\bar{p}) = -\sum_k \bar{p}_k \log \bar{p}_k. \tag{14}$$

Posterior disagreement is quantified by

$$\widehat{I}(y, \theta \mid x, \mathcal{D}) = \mathcal{H}(\bar{p}) - \frac{1}{S}\sum_s \mathcal{H}(p(\cdot \mid x, \theta^{(s)})). \tag{15}$$

Concavity of entropy makes this nonnegative up to numerical round-off. It describes the fitted *conditional approximate posterior*; it is not complete uncertainty about the whole network. No OOD detection benchmark or coverage guarantee is claimed.

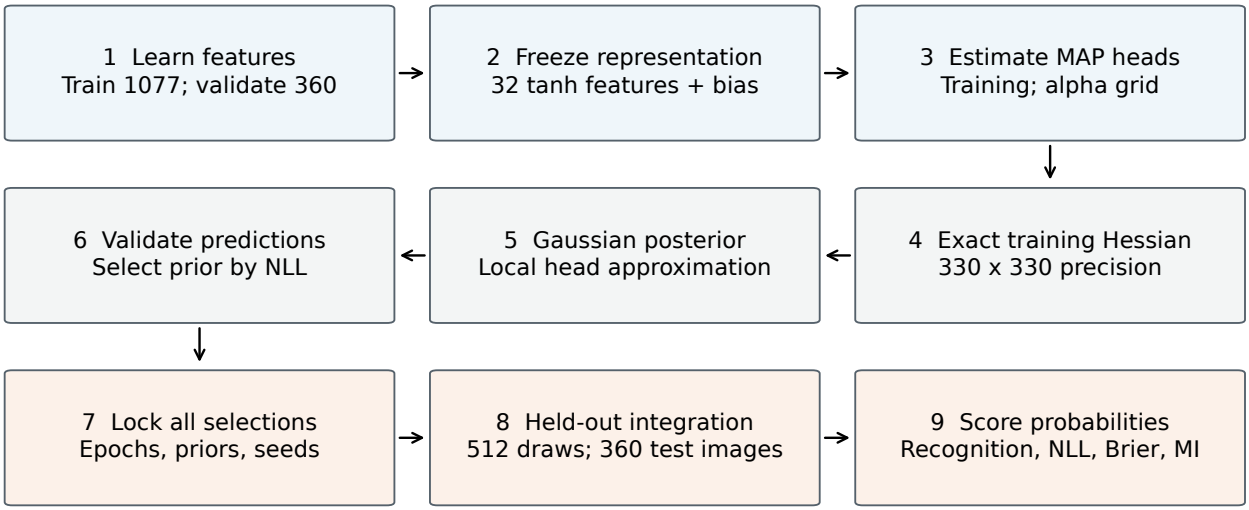


Figure 1: Implemented inference flow. Training supplies features and curvature; validation selects checkpoints and α. Test predictions follow the selection record. Numbers and arrows specify the traversal order.

2.7 Cost and limitations

With $P = KD = 330$, full Hessian storage is $O(P^2)$ and Cholesky factorization is $O(P^3)$. Dense curvature accumulation costs $O(NK^2D^2)$ by direct formulation, sampling costs $O(SP^2)$, and logits cost $O(SMKD)$ for M evaluation images. Feature extraction is additional. These costs scale poorly for very wide heads; full curvature is used here for auditability.

The posterior is local and depends on the feature point estimate, Gaussian prior and model specification. Monte Carlo probabilities have sampling error. The model omits representation uncertainty. Selecting among three priors by validation is not marginalization over α or marginal-likelihood optimization. These project choices are distinct from the literature framework.

3 How AI Implemented the Algorithm

3.1 Modules and equation-to-code mapping

All experiment and reporting code was written by the AI agent. The authoritative workflow is modular Python, without a notebook dependency. Configuration, training, inference, evaluation and presentation are separated.

File	Responsibility and mathematical connection
`data.py`	Images, fixed-range scaling, stratified split indices.
`features.py`	h_ϕ, deterministic training, validation checkpoint.
`laplace.py`	Objective, gradient, Eq. 7, Cholesky draws.
`metrics.py`	Classification metrics, NLL, Brier, calibration bins.
`experiment.py`	Candidates, selection lock, final test predictions.
`figures.py`	Nine evidence figures and one tested toy geometry diagram.
`audit_results.py`	Metrics, checkpoint reload and full-run verification.
`build_report.py`	Numerical LaTeX tables generated from JSON.
`review_diagnostics.py`	Fixed-model posterior sampling repetitions.
`paired.py`	Post hoc paired image scores, bootstrap and bin intervals.

Table 4: Modules reside under `src/bayes_digits/`, scripts under `scripts/`.

The head implementation is original project code, not a call to the paper's `laplace` library. It implements one supported inference variant, not every library feature or the paper's benchmarks.

3.2 Executed pipeline pseudocode

1. Load data, fix stratified indices, save configuration and dataset hash.
2. Fit Gaussian NB smoothing candidates on training images; select by validation NLL.
3. For each seed, train the image network for up to 120 epochs; retain the checkpoint with minimum validation cross-entropy.
4. Freeze features. For each $\alpha \in \{0.1, 1, 10\}$, refit a MAP head, check optimizer convergence and compute training-only posterior curvature.
5. Evaluate MAP and Laplace candidates on validation images; choose their priors separately by minimum validation NLL.
6. Finish all seeds and write `selection_locked.json`. Only then transform test images and predict.
7. Evaluate primary models and the two predeclared frozen-prior ablations. Save test probabilities, indices, labels, confusion matrices and metrics.
8. Generate plots and LaTeX tables; reload checkpoints and rerun the pipeline.

3.3 Checks, repairs and evidence

Thirteen tests passed in the clean environment. The original nine verify portable JSON evidence bytes, split coverage/disjointness, analytic derivatives against independent PyTorch autograd, precision-to-covariance sampling, normalization and serialization, known metric values and bin boundaries, deterministic tiny-batch optimization, and rejection of invalid priors. The binary example additionally checks the exact MAP and logit-contrast covariance. Four added tests check elementwise class-major Hessian indexing and its prior-only common-logit direction, MI against the direct entropy calculation, paired-score identities and Wilson intervals at sparse/empty-bin boundaries. This connects Eq. 7 and predictive integration to independently constructed checks. The tiny-batch test achieves perfect training classification and loss below 0.02 on 20 images; repeated seeded feature training matches exactly.

The first test run failed while creating a Windows system temporary directory; the serialization test had not executed. Moving pytest temporary files inside the project resolved the access issue. A small end-to-end run with three feature epochs executed before the full run. It selected its own models with validation and evaluated after its selection lock; its test results were not used to change the already specified full configuration. The audit then reloaded all primary checkpoints, regenerated recorded metrics, and repeated the full pipeline. All 18 saved arrays matched bit for bit in this environment. Prompt 02 corrected loss logging: both training and validation NLL are now evaluated after the completed epoch's optimizer update. Rerunning preserved all 18 original prediction/label/index/MI arrays bit for bit against Git snapshot `a28a563`; only the training-loss history and execution provenance changed. The original results remain recoverable from that snapshot.

4 Experiment Settings and Results

4.1 Dataset provenance and protocol

The data are the `sklearn.datasets.load_digits` copy of the UCI optical handwritten digits dataset's 1,797-image *test subset* [5, 6]. The full UCI collection has a separate larger training file, which this project does not use. These experiments repartition the bundled subset; they are not results on UCI's original official protocol. UCI records CC BY 4.0 usage conditions and a dataset DOI.

The images are available offline through scikit-learn, and the dataset is small enough to verify full posterior matrices. Recognition uses flattened images without augmentation, pretrained weights, PCA or fitted standardization. Division by 16 follows the measurement range and fits no preprocessing parameters on held-out images.

A stratified split holds out 20% of all images with random state 3024. A second stratified split holds out 25% of the remainder using the same state. Integer rounding gives 1,077 training, 360 validation and 360 test images. Indices and per-class counts are saved. Test support ranges from 35 to 37 images per class. Figure 2 uses those saved indices.

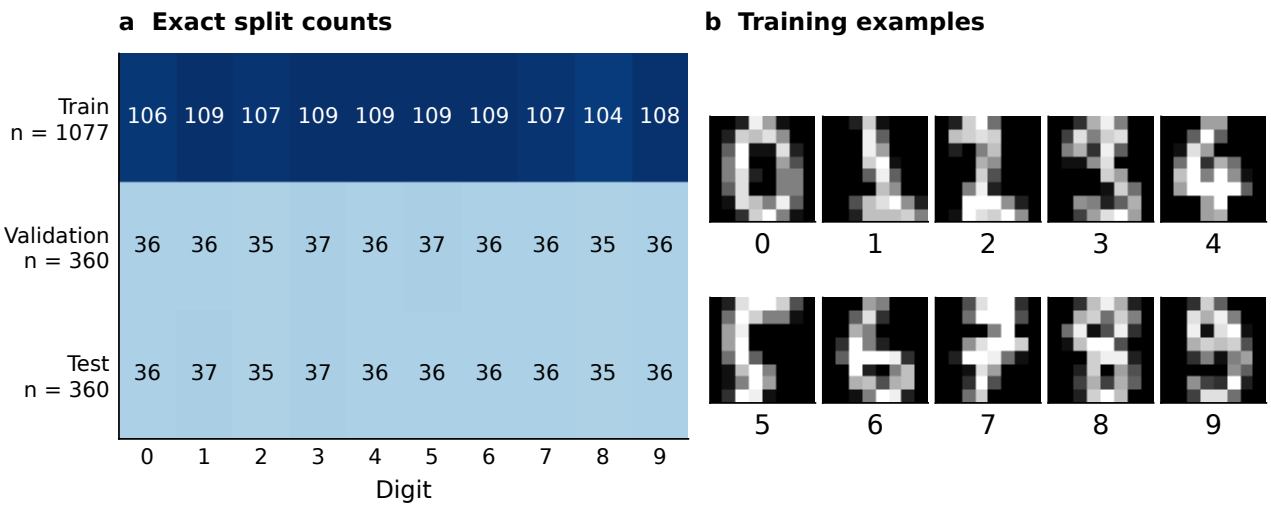


Figure 2: Class counts by split and the first training image of each digit, enlarged in two rows. The classes are nearly balanced. The range and nearest-neighbor display are fixed; examples are chosen deterministically.

The evaluation unit is an image in this fixed subset. Writer identifiers are not provided by the bundled loader, so writer-disjoint splitting is not demonstrated. A new-writer deployment claim is unsupported. Three seeds describe optimization variability conditional on one split; they do not turn 360 held-out images into 1,080 independent test examples.

4.2 Research questions and controls

The pre-execution questions were whether conditional last-layer inference retains useful accuracy, whether posterior averaging improves probability scores over a matched point predictor, and whether posterior correlations matter. The frozen-prior MAP ablation isolates averaging at the same learned representation and prior. The diagonal ablation isolates curvature structure with the same mean, selected prior and sample count.

Gaussian NB is a simple statistical baseline with conditionally independent Gaussian pixels and empirically estimated class priors:

$$p(y = k \mid x) \propto \frac{N_k}{N} \prod_{d=1}^{64} \mathcal{N}(x_d; \mu_{kd}, \sigma_{kd}^2 + \epsilon), \quad \epsilon = v \max_d \mathrm{Var}_{\mathrm{train}}(x_d). \tag{16}$$

Class means and maximum-likelihood variances use training images only; validation selects smoothing v. Normalized log-joint scores produce probabilities. The selected $v = 0.1$ stabilizes near-constant pixels. Spatial pixel correlations challenge its assumptions. The separately tuned MAP head is a discriminative comparison. All methods use identical partitions and a validation-NLL selection rule. Equal architecture or search difficulty between NB and the neural feature model is not assumed.

4.3 Exact configuration and environment

Setting	Executed value
Input, classes	64 pixels in $[0, 1]$, ten classes
Feature network	Linear $64 \to 32$, tanh, linear $32 \to 10$ training head
Neural parameter count	2,410 including biases
Initialization seeds; split seed	11, 22, 33; 3024
Feature training	Adam, full training batch, learning rate 0.01
Feature decay; epoch budget	10^{-4}; 120 epochs
Checkpoint criterion	Minimum validation mean cross-entropy
Conditional head inference	SciPy L-BFGS-B, float64, max 2,000 iterations
Stopping options	`ftol` 10^{-14}, `gtol` 10^{-7}; gradient check $< 10^{-3}$
Prior precision grid	0.1, 1, 10; biases included
Gaussian NB smoothing grid	10^{-9}, 10^{-3}, 10^{-1}
Posterior prediction	512 samples, full 330×330 Hessian
Calibration bins	Ten equal-width top-label confidence bins
Actual device; CPU threads	CPU; four PyTorch threads
Python; PyTorch	3.12.14; 2.11.0+cu128
NumPy; SciPy; scikit-learn	2.5.2; 1.18.1; 1.9.1
Operating system	Windows 11, build 26300
Detected GPU	NVIDIA GeForce RTX 5060 Ti; available but unused

Table 5: Configuration and observed environment. GPU availability does not imply GPU execution. Complete provenance is in `environment.json`.

CPU computation was chosen because this small full-batch problem and its dense conditional Hessian are inexpensive. The review training/selection/evaluation command took 6.45 seconds, excluding setup, tests, plotting and compilation. This is one observed wall-clock duration, not a controlled hardware benchmark. Existing PyTorch/scientific packages were reused read-only through a local environment path in the original execution. Prompt 05 installed an independent Python 3.12 environment using pinned dependencies and the CPU wheel of PyTorch 2.11.0. It reproduced all 18 arrays and loaded every checkpoint exactly; the README gives portable clean-clone commands. Original timing and environment fields are retained; only the public command path is redacted with a hash ledger.

Seed	Feature epoch	MAP α	LA α	LA validation NLL
11	120	0.1	1	0.1516
22	120	0.1	1	0.1519
33	120	1	1	0.1648

Table 6: Choices locked before full-run test predictions. All feature checkpoints were selected at epoch 120. Laplace chose $\alpha = 1$ for every seed. Gaussian NB selected smoothing 0.1.

The validation minimum at the budget boundary indicates that loss was still improving over this range. The prespecified budget is reported rather than increased in response to test results. Each prior candidate's validation NLL, iterations and gradient residual is preserved. Full test probabilities permit later audits without changing model choices.

4.4 Evidence preservation

The saved configuration, split indices, dataset hash, selection lock and environment record identify the protocol. `metrics.json`, `predictions.npz` and `scalar_metrics.csv` preserve numerical evidence. Model files store the selected representation and conditional posterior. Plot and table generators consume these saved files rather than embedded hand-typed measurements.

4.5 Metric definitions and overall results

For $M = 360$, accuracy is the proportion of correct argmax labels. Class precision is $TP/(TP + FP)$, recall is $TP/(TP + FN)$, and $F_1 = 2PR/(P + R)$, with zero when the relevant denominator vanishes. Macro metrics

average the ten class-specific values. Laplace probability metrics use posterior-averaged probabilities:

$$\text{NLL} = -\frac{1}{M}\sum_{i=1}^{M}\log \bar{p}_{y_i}(x_i), \tag{17}$$

$$\text{Brier} = \frac{1}{M}\sum_{i=1}^{M}\sum_{k=0}^{K-1}(\bar{p}_k(x_i) - \mathbb{1}\{y_i = k\})^2. \tag{18}$$

Logs are natural, so NLL is in nats. NLL clips probabilities below 10^{-15}. Brier is summed over classes, not divided by ten; its range is $[0, 2]$. NLL of averaged probabilities differs from average sampled-head NLL.

For top-label calibration, confidence is $c_i = \max_k \bar{p}_k(x_i)$. Let $B_b = \{i : b/10 \le c_i < (b+1)/10\}$, with the final bin also including $c_i = 1$. Then

$$\text{ECE} = \sum_{b=0}^{9}\frac{|B_b|}{M}|\text{accuracy}(B_b) - \text{confidence}(B_b)|. \tag{19}$$

Empty bins contribute zero. This is a usual top-label calibration convention [7]; ECE depends on binning and is not a proper scoring rule. NLL and Brier carry more weight in evaluating probability quality.

Method	Accuracy (%)	Macro-F1	NLL	Brier
Gaussian NB	91.94	0.920	0.630	0.134
MAP	97.78 ± 0.28	0.978 ± 0.003	0.099 ± 0.003	0.040 ± 0.002
Laplace	97.69 ± 0.16	0.977 ± 0.002	0.151 ± 0.003	0.055 ± 0.001

Table 7: Neural results are mean ± sample SD across three seeds. The deterministic NB baseline was evaluated once. SD describes seed variability on a shared test set, not a confidence interval.

Method	Macro-precision	Macro-recall	ECE
Gaussian NB	0.924	0.920	0.063
MAP	0.978 ± 0.002	0.978 ± 0.003	0.017 ± 0.008
Laplace	0.977 ± 0.001	0.977 ± 0.002	0.080 ± 0.003

Table 8: Additional classification and calibration metrics. Lower ECE is better under the specified ten-bin estimator.

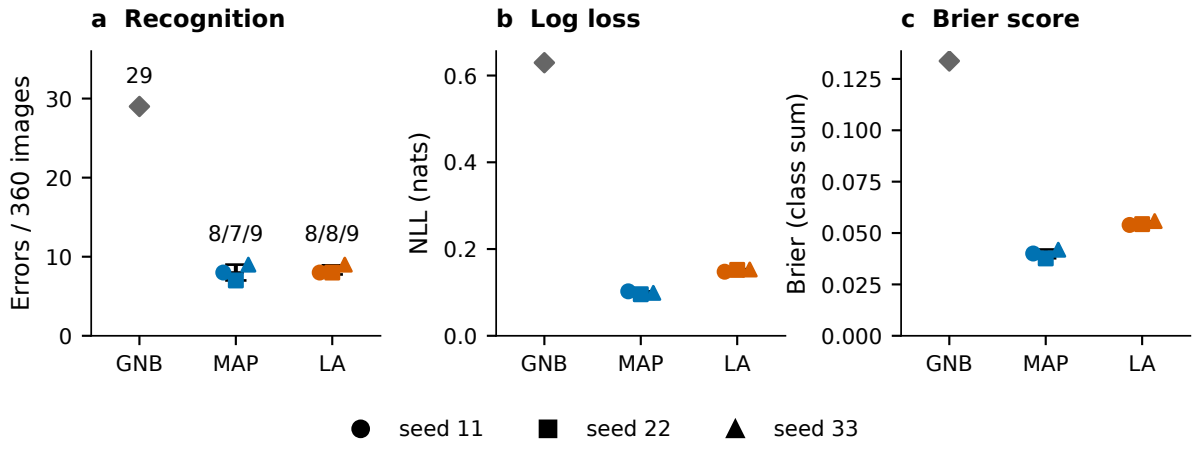


Figure 3: Recognition error counts, NLL and class-summed Brier score; lower is better. Horizontally offset shapes show each neural seed, black bars show mean ± sample SD, and GNB has one observation. All use the same 360 test images. Axes start at zero. Neural methods make fewer errors than GNB, but MAP has better probability scores than Laplace (LA).

The neural methods recognize digits more accurately than NB. MAP and Laplace have similar accuracy, but MAP has lower NLL and Brier; Laplace ECE is higher. The experiment therefore does not support improved probability quality from this approximation. Accuracy alone would miss that result. Small accuracy differences involve only a few images. No significance test or statistical equivalence claim is made.

4.6 Post hoc paired image-level analysis

Prompt 05 quantified the matched-prior comparison using the saved probabilities. The estimand is the mean Laplace-minus-matched-MAP score across the three *fixed fitted models* on this split. For additive scores, let $d_i = \frac{1}{3}\sum_{r=1}^{3}[\ell_{ir}^{\mathrm{LA}} - \ell_{ir}^{\mathrm{matched}}]$; the estimate is $M^{-1}\sum_i d_i$. The analysis plan was recorded before this supplementary calculation, after the primary outcomes were already known. It jointly resamples $M = 360$ image positions with replacement, preserving pairing across both methods and all seeds, for 5,000 repetitions (seed 20261012). The reported endpoints are empirical 2.5th/97.5th percentiles; they are a transparent descriptive bootstrap, not bias-corrected or confirmatory coverage. ECE is nonlinear, so bins and ECE are recomputed within each resample and then averaged over the fixed models. This follows paired-resampling logic [9].

Score difference	Estimate	Paired percentile interval
Error (percentage points)	0.00000	[-0.27778, 0.27778]
NLL (nats)	0.05389	[0.03994, 0.06612]
Brier (class sum)	0.01358	[0.00823, 0.01878]
ECE (ten bins)	0.05536	[0.04162, 0.05959]

Table 9: Exploratory Laplace-minus-matched-MAP differences and conditional 95% paired image-bootstrap percentile intervals. Scores use the same 360 images; error uses percentage points. Positive proper-score differences favor matched MAP.

The error-count difference is zero for each model, but the identities of errors can differ; the interval does not establish equivalence. NLL and Brier penalties remain positive throughout these conditional intervals. These describe finite-image variation assuming exchangeable images; unknown writer dependence is not modeled. They exclude model retraining and fresh posterior draws. Initialization SD, Monte Carlo repetition SD and these intervals therefore answer different questions. ECE remains bin-dependent and biased in finite samples, especially in sparse bins; its bootstrap should be interpreted as exploratory estimator variability rather than a definitive calibration guarantee.

4.7 Class-level results and feature training

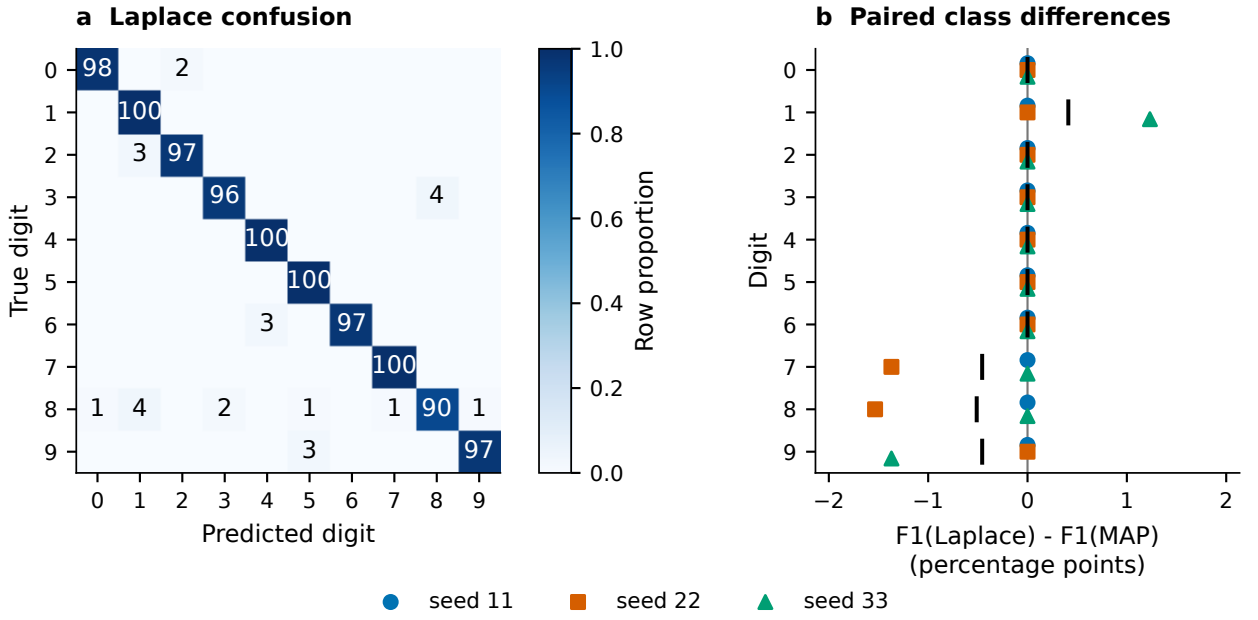


Figure 4: Laplace confusion is normalized by true-class support within each seed, then averaged. Cell labels are percentages rounded to the nearest integer; entries below 0.5% are unlabeled. The color scale uses proportions. The right panel plots paired F_1(Laplace) − F_1(MAP) in percentage points for each digit and seed against separately selected MAP. Shapes identify seeds; black vertical dashes are descriptive means, not intervals. The symmetric signed scale preserves all negative and positive differences; Table 10 gives raw Laplace F1. Digit 8 is the weakest class. There are 360 unique test images.

Digit	Test support	Precision	Recall	F1
0	36	0.991	0.981	0.986
1	37	0.941	1.000	0.969
2	35	0.981	0.971	0.976
3	37	0.982	0.964	0.973
4	36	0.973	1.000	0.986
5	36	0.964	1.000	0.982
6	36	1.000	0.972	0.986
7	36	0.991	1.000	0.995
8	35	0.960	0.905	0.931
9	36	0.991	0.972	0.981

Table 10: Mean Laplace per-class metrics across three seeds. Support counts unique images, not three times that number.

Digit 8 has the weakest class F1 (0.931), with recall 0.905. Its averaged mistakes are spread across several labels, including 1 and 3. Digit 1 has perfect recall but lower precision because other digits are sometimes assigned to it. Thus recall alone does not establish reliable predictions of a label.

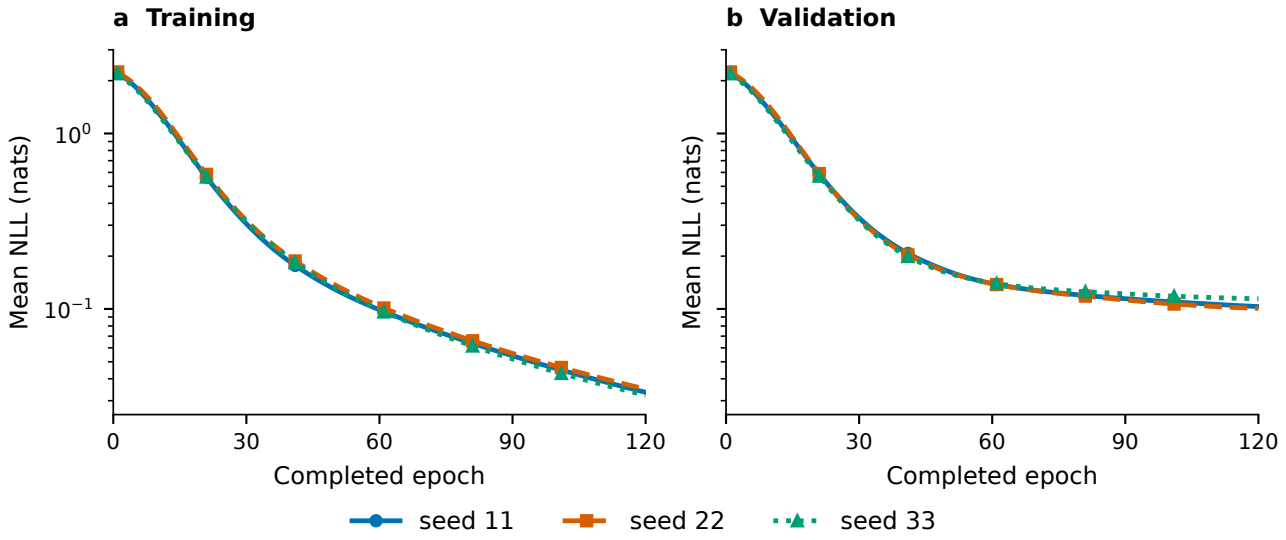


Figure 5: Recorded feature-training and validation mean NLL on base-10 logarithmic axes. Both panels share the same positive NLL limits and are evaluated after each epoch's optimizer update, using the original training head before conditional MAP refitting. Line styles and markers identify seeds. Loss continues to improve over the fixed budget; these curves do not establish asymptotic convergence.

Both losses decrease through the prescribed budget. The curves do not prove asymptotic feature-network convergence. Conditional head optimization is a separate convex problem with checked gradient residuals; posterior inference has no stochastic training trajectory in this implementation.

4.8 Calibration, uncertainty and ablations

Several populated Laplace reliability bins lie above the identity line: confidence is lower than observed success frequency. This is compatible with underconfidence in this run, but many low-support intervals are wide. Higher NLL/ECE alone does not prove underconfidence, and eight errors limit precision. Bin support and uncertainty prevent treating tiny bins as reliable evidence. Many correct images have low posterior disagreement, while errors span a broader range; within-group normalization exposes both distributions despite their unequal sample counts. Overlap prevents a guaranteed error detector. The representative seed was fixed before training.

Matched MAP has lower NLL than full Laplace with essentially identical aggregate accuracy, isolating the effect of averaging rather than a different prior. Diagonal Laplace increases NLL further. Keeping correlations improves this approximation relative to the diagonal variant, but not relative to the point estimate. Removing correlations can destroy cancellation of uncertain common-logit directions. This mathematical explanation is compatible with the results, not a separately measured causal decomposition.

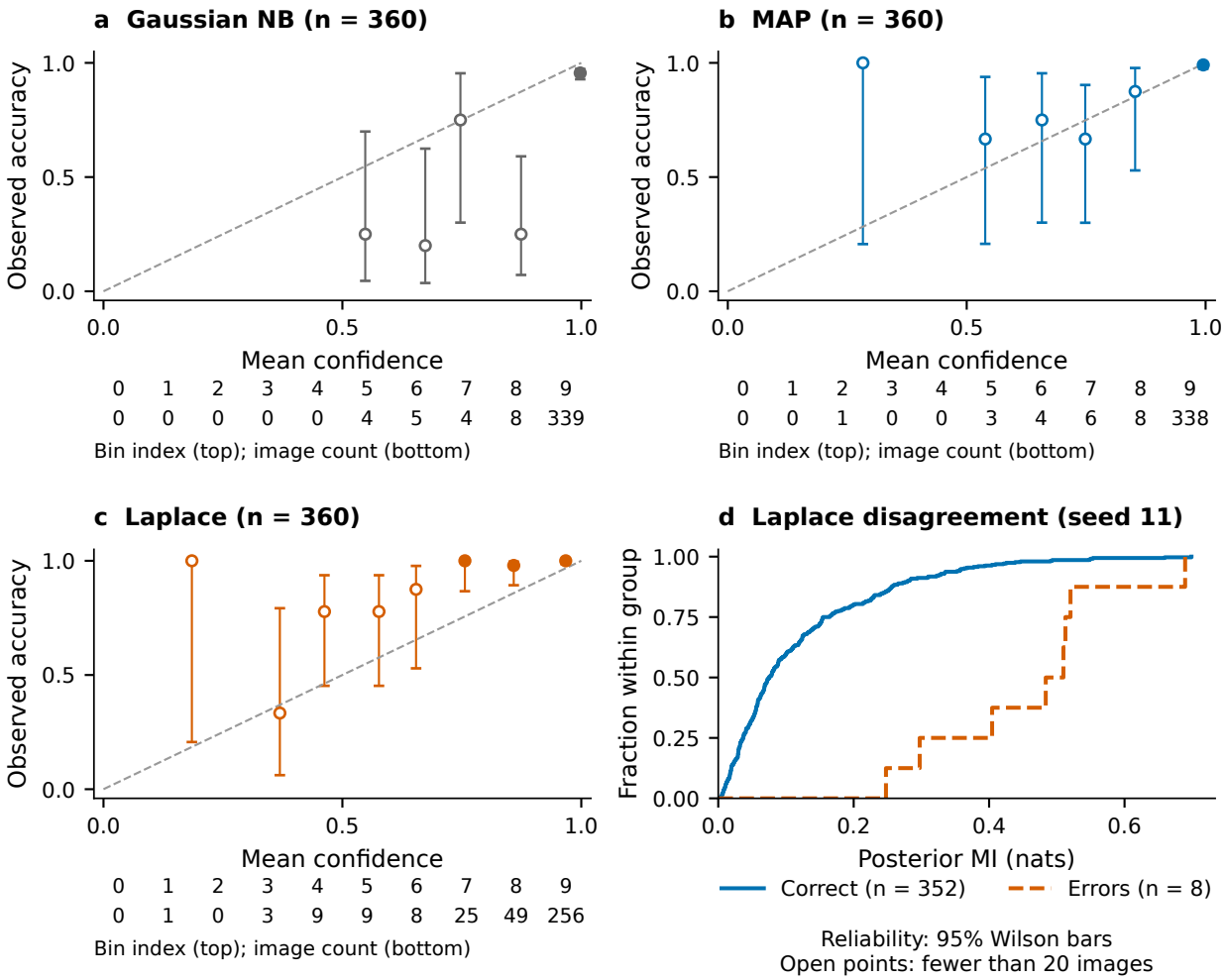


Figure 6: Reliability and posterior mutual information for prespecified seed 11. Panels a–c use ten equal-width confidence bins, unconnected occupied-bin points and their exact support counts (bin indices 0–9). Vertical bars are marginal 95% Wilson score intervals for bin accuracy; open points flag fewer than 20 images, a display threshold rather than a rule for deleting bins. These intervals condition on bin membership and approximate independent Bernoulli outcomes; they do not quantify uncertainty in mean confidence or provide a simultaneous calibration band. Empty bins have no point or interval. Panel d: empirical cumulative distributions normalized separately over 352 correct and eight erroneous predictions. Errors often have greater disagreement, but the distributions overlap and eight errors provide limited precision.

Frozen-prior variant	Accuracy (%)	NLL	ECE
Matched MAP	97.69 ± 0.16	0.097 ± 0.002	0.025 ± 0.002
Laplace	97.69 ± 0.16	0.151 ± 0.003	0.080 ± 0.003
Diagonal Laplace	97.69 ± 0.42	0.229 ± 0.0002	0.147 ± 0.005

Table 11: Features and Laplace-selected prior are held fixed. Matched MAP removes averaging; diagonal Laplace drops off-diagonal precision terms before inversion. This differs from taking the diagonal of the full covariance.

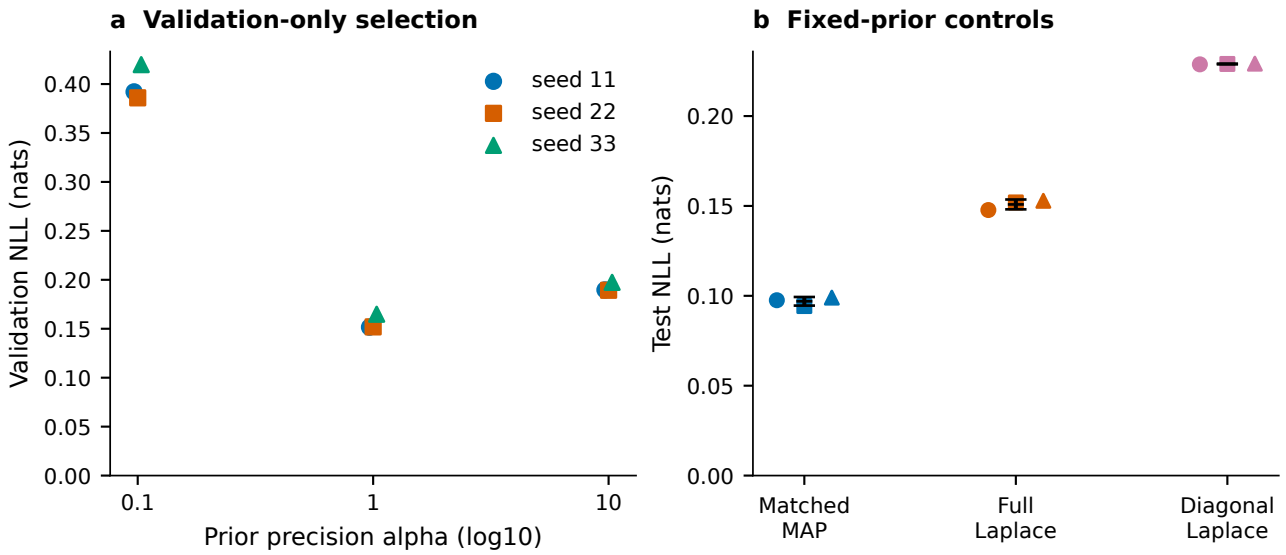


Figure 7: Left: the three discrete validation-only prior candidates, shown as unconnected seed points with small horizontal offsets for visibility. No interpolated optimum is estimated. Right: predeclared frozen-prior test NLL and seed SD, with offset seed points. $\alpha = 1$ gives the lowest validation NLL in this grid; full Laplace has lower test NLL than diagonal precision but higher NLL than matched MAP. Test scores were not used to retune the prior.

Both $\alpha = 0.1$ and $\alpha = 10$ give worse Laplace validation NLL than $\alpha = 1$ for every seed. This demonstrates prior sensitivity within the specified grid, not a globally optimal precision or an empirical-Bayes result.

4.9 Post hoc Monte Carlo integration diagnostic

Prompt 02 identified unmeasured sampling variation as a threat to interpreting finite-sample predictions. The diagnostic held all selected features, heads, priors and splits fixed; it did not perform model selection. Before executing it, `configs/review_diagnostics.json` specified sample counts 128/512/2048 and eight independent repetitions per model/count, giving 72 probability arrays on the same 360 test images. Sampling seed is $2002000 + 10000r + 10S + j$ for feature seed r, sample count S and repetition $j \in \{0, \ldots, 7\}$. Repetition SD measures numerical integration variation conditional on a fitted posterior, not test-set or retraining uncertainty.

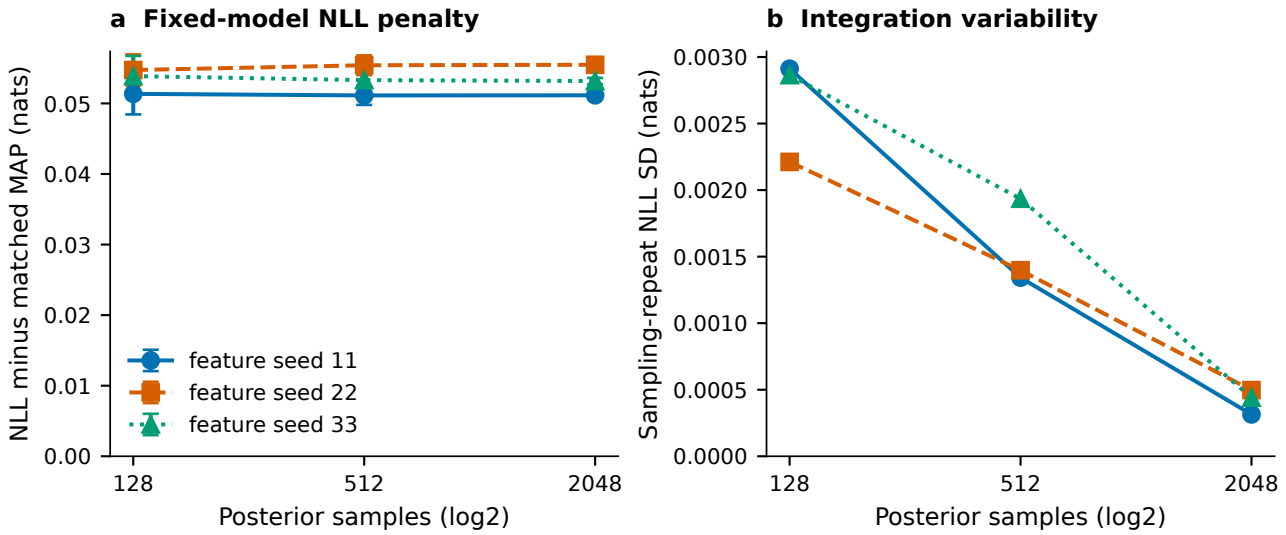


Figure 8: Post hoc fixed-model sampling diagnostic. Left: mean NLL penalty against matched MAP with sampling-repeat SD, eight repetitions per point. Right: the same SD alone. The penalty persists at 2048 samples while numerical variation decreases; sampling noise alone does not account for the observed gap. Models were not retuned and primary 512-sample results remain unchanged.

At 512 draws, within-model NLL SD across eight repetitions ranges from 0.0013 to 0.0019 nats. At 2048 draws,

mean Laplace-minus-matched-MAP NLL remains 0.0511–0.0555 nats across the three fitted models. These descriptive results support a persistent probability-score penalty in this setting, rather than attributing it solely to the original random draws. Independent repetitions sample the same approximate Gaussian, so this check cannot establish the accuracy of the Laplace posterior approximation itself.

4.10 Failure cases, interpretation and limits

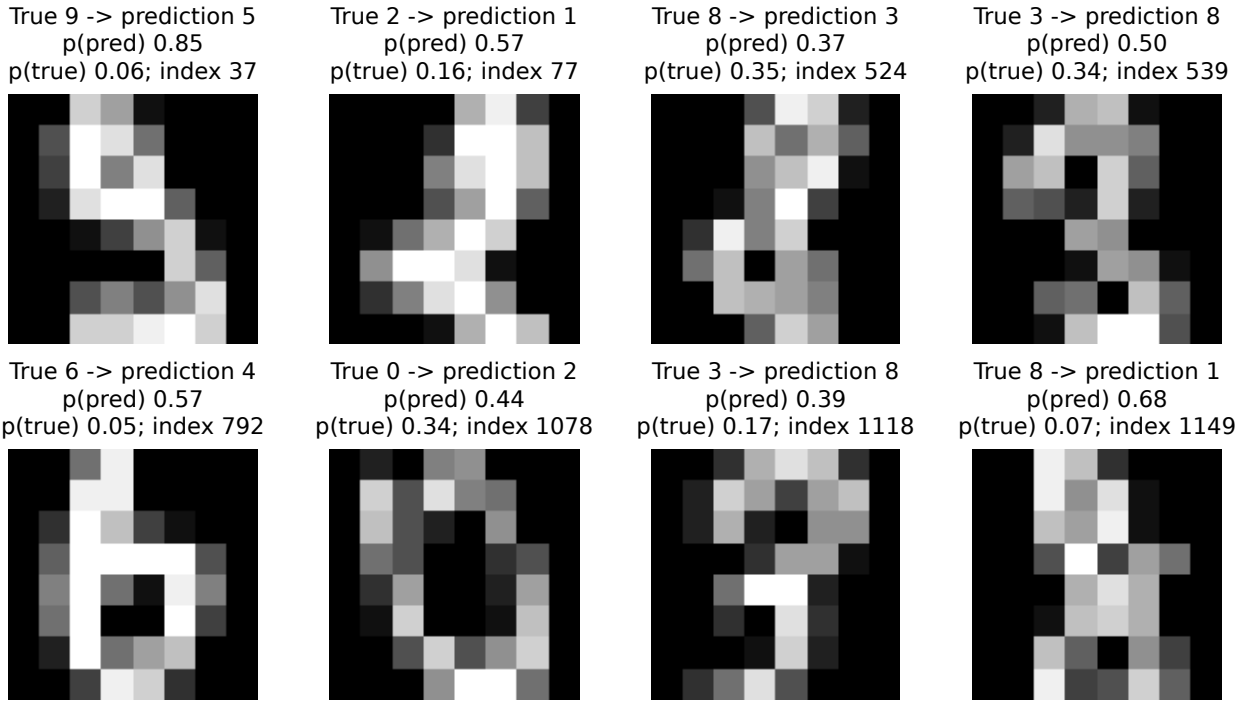


Figure 9: All eight Laplace errors for seed 11, ordered by original dataset index. Titles show true digit, predicted digit, both relevant posterior probabilities and original index. Native 8×8 pixels are displayed without smoothing.

Several mistakes involve ambiguous low-resolution strokes; that interpretation is descriptive, not a measured explanation of image quality. The 9-to-5 case has confidence 0.85, showing that a Bayesian predictor can still make a relatively confident mistake. The 8-to-3 case has much lower confidence. Selecting only favorable examples would conceal this variation.

Why might probability scores deteriorate? The matched-prior comparison demonstrates the effect of this implemented posterior average, while the MC diagnostic shows that a persistent gap is not explained solely by the original finite draws. The local Gaussian and fixed representation can still misdescribe conditional uncertainty; those mechanisms were not separately measured. Softmax is nonlinear, so averaging logits or using the MAP head is generally different from averaging probabilities. Convexity of $-\log$ makes NLL of the predictive mixture no greater than average sampled-head NLL, but gives no ordering against MAP NLL. Averaging can dilute probabilities assigned to correct labels in an already accurate point predictor. The coarse prior grid and feature scale influence posterior dispersion; validation selection does not guarantee test improvement. These are bounded hypotheses, not a measured causal decomposition or proof that Bayesian inference generally harms calibration.

Answers to the research questions. Conditional Laplace inference retains high recognition accuracy in this experiment. It does not improve the primary proper probability scores over independently selected MAP or the matched-prior MAP control. Full covariance improves NLL over diagonal precision, supporting attention to covariance structure rather than treating approximations as interchangeable.

Interpretation boundary. One small, nearly balanced dataset, one split, one architecture and three initialization seeds cannot establish a general ranking of Bayesian classifiers. The Gaussian is local and omits representation uncertainty. The prior grid is coarse and feature-scale dependent. Checkpoint and prior selection reuse the same validation partition; there is no nested validation estimate. The fixed-model diagnostic quantifies integration variation at three sample counts. The separate paired bootstrap addresses conditional image resampling; neither covers representation uncertainty, unknown writer dependence or new splits. Writer independence, natural-image recognition, corruption and OOD detection were not tested.

Prospective extensions include a wider validation-selected prior grid, empirical-Bayes tuning, writer-separated data or a second dataset. They were not executed and do not justify revising the current conclusion using test outcomes. The negative calibration result belongs in the evidence.

Reproducibility. `audit.json` verifies regenerated metrics, checkpoint reloads and an identical full rerun. Tables read saved JSON; figure provenance binds plots to source code and evidence. The original execution hashes are preserved separately from revised plotting code. These checks establish local reproducibility, not bitwise portability. Unlike the large-network benchmarks in Laplace Redux [1], this study uses a small head, frozen features and validation-grid prior selection. The negative probability-quality result therefore evaluates this adaptation and protocol, rather than refuting the paper's broader framework. The practical lesson is to compare probability scores and numerical sensitivity alongside recognition accuracy before drawing conclusions about uncertainty quality.

5 What I Have Learnt from This AI Assignment

This assignment taught me that using an AI coding agent well is not just about asking it to write code. At the start, I thought the main challenge would be getting the agent to produce something that actually ran. But I quickly realized the harder part was giving it enough structure to keep the project on track. I used AGENTS.md to set the long-term rules: scientific integrity, reproducibility, mathematical rigor, experiment requirements, and report quality. Then I used smaller, task-specific Markdown prompts to break the work into stages: finding the algorithm, implementing it, running experiments, reviewing the results critically, and preparing the final report. That separation mattered. The permanent rules told the agent how the project should always behave, while the task prompts told it what to do next. I also learned that vague instructions lead to vague results. Clear requirements, constraints, and acceptance criteria made a real difference.

I also saw how specialized AI skills can raise the quality of the work. Instead of depending only on the model's general knowledge, I gave it skills for literature research, machine learning, PyTorch implementation, experimental design, statistical analysis, scientific visualization, and academic writing. The prompts explained what needed to be done, while the skills gave guidance on how to do it. The critical-review stage was especially useful. It pushed the agent to look back at its own results, find weak spots, improve the figures, and strengthen the technical explanations. Without that stage, I probably would have accepted the first report, even though it could have been much better.

Technically, the assignment deepened my understanding of Bayesian classification and uncertainty estimation. By studying the last-layer Laplace approximation, I saw how priors, likelihoods, and Bayes' theorem connect to posterior inference. I also got a clearer picture of maximum a posteriori estimation, Hessian matrices, Gaussian posterior approximations, and Monte Carlo prediction. One point that I initially misunderstood was that computing an exact conditional Hessian does not make the posterior itself exact. The Laplace approximation is still a local Gaussian approximation, and it does not fully capture uncertainty in the learned feature representation.

Another lesson was that a more sophisticated algorithm does not always produce better results. The Laplace classifier reached about 97.69% recognition accuracy, but its negative log-likelihood was higher than that of the MAP classifier. That result was humbling. It showed me that accuracy alone is not enough, especially for probabilistic models. Negative log-likelihood, Brier score, and calibration error tell us something important about whether the predicted probabilities can be trusted. I also learned to value controlled comparisons, repeated experiments, validation-based model selection, and careful error analysis. Even negative experimental findings can be useful if they are supported by evidence and explained clearly.

Overall, this assignment changed how I think about AI in programming and research. The quality of AI-generated work depends not only on how strong the model is, but also on how well the user organizes instructions, provides relevant skills, and defines verification standards. An AI agent can automate a lot of implementation, experimentation, and documentation, but I still need to understand the underlying concepts, question the methodology, and interpret the results. I now see AI as a powerful assistant, not a replacement for human judgment. Its usefulness depends on clear direction, reliable verification, and critical thinking.

6 Webpage Link of the Source Codes

The public source repository is:

https://github.com/Nvb-flipped/bayesian-handwritten-digit-recognition. It contains source code, tests, configurations, experimental evidence, original prompts, skill documentation and this report. GitHub confirms public visibility. The populated public wiki is:

https://github.com/Nvb-flipped/bayesian-handwritten-digit-recognition/wiki. Six technical pages are synchronized from `docs/wiki/`; equations, results, figures and navigation were verified on GitHub.

Local reproduction

After installing the pinned requirements, execute from the project root:

```
python scripts/reproduce.py           # Windows / POSIX
cd report
tectonic -X compile --untrusted main.tex
```

Compile `report/main.tex` with its tables and figures available. The verified compiler is Tectonic 0.17.0; no private plugin is required by the portable recipe. README includes standalone installation and compilation commands. The generated PDF is rendered and reviewed page by page.

References

[1] E. Daxberger, A. Kristiadi, A. Immer, R. Eschenhagen, M. Bauer and P. Hennig. *Laplace Redux—Effortless Bayesian Deep Learning*. NeurIPS 34, 2021. https://papers.nips.cc/paper/2021/hash/a7c9585703d275249f30a088cebba0ad-Abstract.html.

[2] B. Charpentier, O. Borchert, D. Zügner, S. Geisler and S. Günnemann. *Natural Posterior Network: Deep Bayesian Uncertainty for Exponential Family Distributions*. ICLR, 2022. https://arxiv.org/abs/2105.04471.

[3] J. Harrison, J. Willes and J. Snoek. *Variational Bayesian Last Layers*. ICLR, 2024. https://arxiv.org/abs/2404.11599.

[4] D. J. C. MacKay. *Bayesian Interpolation*. Neural Computation 4(3):415–447, 1992. https://doi.org/10.1162/neco.1992.4.3.415.

[5] E. Alpaydin and C. Kaynak. *Optical Recognition of Handwritten Digits*. UCI dataset, 1998. https://doi.org/10.24432/C50P49. CC BY 4.0.

[6] Scikit-learn documentation. *load_digits*. Accessed 8 October 2026. https://scikit-learn.org/stable/modules/generated/sklearn.datasets.load_digits.html.

[7] C. Guo, G. Pleiss, Y. Sun and K. Q. Weinberger. *On Calibration of Modern Neural Networks*. ICML, PMLR 70:1321–1330, 2017. https://proceedings.mlr.press/v70/guo17a.html.

[8] T. Kassis, V. Agarwal, Y. He, D. Patel and A. M. Brueckner. *Scientific Agent Skills: A Library of Procedural Knowledge for Research Agents*. ArXiv:2609.00065, 2026; record revised 2 September 2026. https://doi.org/10.48550/arXiv.2609.00065.

[9] SciPy developers. *scipy.stats.bootstrap* documentation. Paired resampling and percentile intervals. Accessed 12 October 2026. https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.bootstrap.html.